Planning at the Speed of Control

Profiling and fusing world-model robot planning into a single GPU kernel

Antoine Nguyen · **24433599**
Project presentation · [Faculty / School], University of Technology Sydney
Supervisor: [supervisor name] · October 2026

Overview

- LeWM is a 15M-parameter world model that plans by evaluating thousands of predicted action outcomes. One plan takes 622 ms on an H100 GPU.
- Research question: where is the planning latency spent, and how much can be removed without retraining the model or reducing task success?
- Method: compute profiling of 11,942 timed calls, a fitted latency-scaling model, solution design, and paired statistical evaluation on 900 episode outcomes.
- Result: latency was dominated by execution overhead rather than computation. A single persistent GPU kernel per plan reduces latency to 46.5 ms with no detectable change in task success.

622 ms
reference planner
(one robot, H100)

46.5 ms
LeWM-MK,
one GPU launch

13.4×
faster than the
reference

–0.3 pts
success vs reference,
300 paired episodes

Medians of end-to-end planning latency; 95% CI of the speed-up 13.0–13.5×

Motivation

- A world model predicts future states; a planner evaluates many candidate action sequences with it and executes the best.
- A controller must finish each plan within its control period: 100 ms at 10 Hz, 50 ms at 20 Hz, for sensing, planning and acting together.
- LeWM made world-model planning cheap by shrinking the model to 15M parameters. Its released planner still takes 622 ms per plan.
- The arithmetic content of a plan corresponds to milliseconds of GPU time; the remaining latency is not attributable to computation.

15M
parameters in the whole model

9,000
rollouts per reference plan

900
dependent layer evaluations per plan

16–53 s
per plan for larger world models

Aim and objectives

01 Profile: build a stage-by-stage compute profile of the reference planner from measured data, and identify the bottleneck.

02 Design and implement: remove the bottlenecks without retraining the model or shrinking the planner's search budget.

03 Evaluate: latency and task success against the reference and a strong optimised baseline, on the same GPU, with paired tests.

04 Deploy: a planner that plugs into the existing harness, and one program that regenerates every result from the data.

Data analytics and AI. The object is an AI system (a self-supervised world model and its search algorithm); the method is an analytics pipeline over its measured behaviour: collection, preparation, pattern discovery with a fitted latency model, statistical evaluation and deployment.

Background and research gap

World models that plan

JEPA models (I-JEPA, DINO-WM, V-JEPA 2, LeWM) predict in a learned representation and plan by search. Planning time is reported coarsely and only ever cut by changing the model.

Sampling-based search

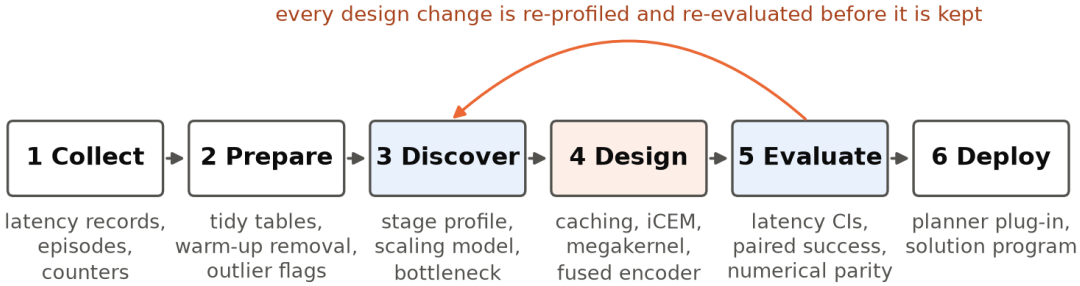
CEM samples, keeps the best, refits, repeats. iCEM shrinks the population and reuses the best: 2.7–22× fewer samples. But fewer samples need not mean less time (CEM-GD).

Fast GPU inference

Small models at batch 1 are bound by launch overhead and data movement. Remedies: CUDA Graphs, compilers, FlashAttention, and megakernels: one GPU program for a whole computation.

Research gap. No prior work has profiled where a world-model planner's latency is spent, measured whether sample-efficient search saves time in practice, or fused a whole planning loop, with a neural encoder, rollouts, scoring and elite selection, into one GPU program.

Methodology



Data

11,942 timed policy calls (61 one-robot episodes × 3 systems + 4 probes), 900 episode outcomes, telemetry, component timings.

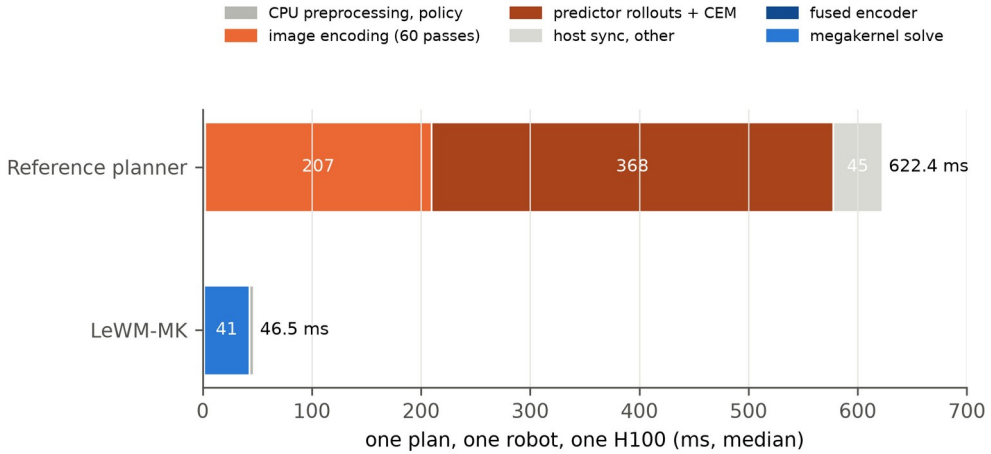
Analysis

Warm-up removal and outlier flags; stage decomposition; a fitted model latency = a + b·N separating fixed overhead from work.

Evaluation

Medians with bootstrap CIs; 300 paired episodes, McNemar and paired bootstrap; numerical error against FP32.

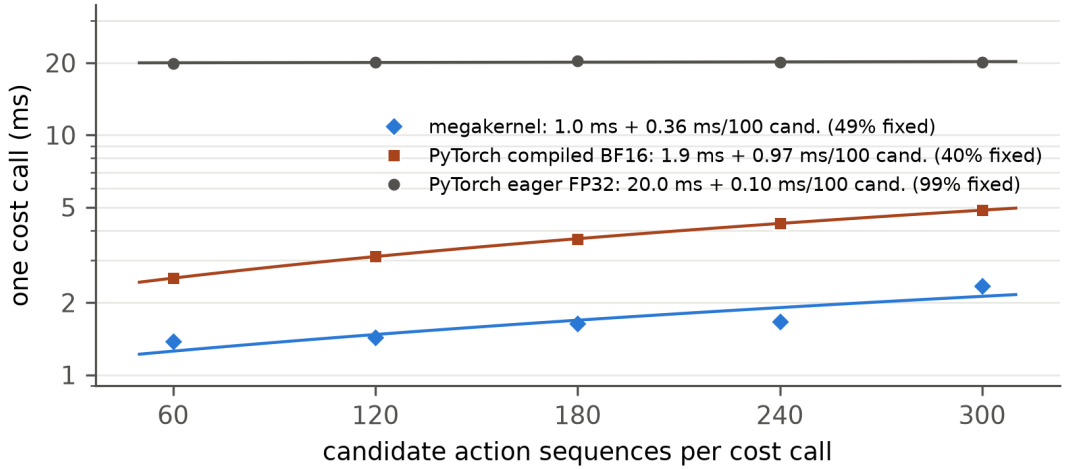
Compute profile of the reference planner



Top bar: reference planner, by stage. Bottom bar: LeWM-MK (Section 6). Stages without a direct measurement attributed by difference.

- 33%: re-encoding the same two images 60 times per plan (telemetry: 60 encoder calls, 2 distinct images).
- 59%: the search, 12.3 ms per iteration, 409 µs per layer for about 3 µs of arithmetic.
- 7%: CPU synchronisation between iterations.
- Under 1%: the policy code around the planner.

Latency scaling analysis



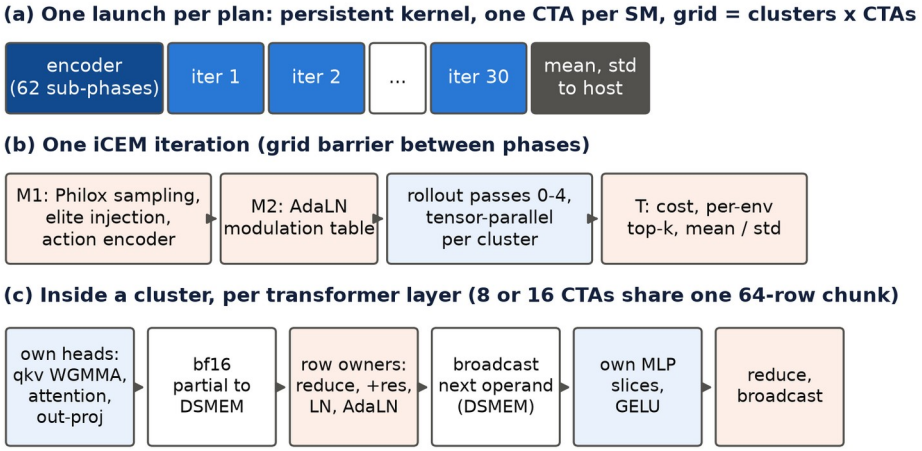
Latency of one scoring call vs number of candidates, with fitted lines a + b·N.

99%
of an eager call is fixed cost

1.01×
predicted iCEM gain, eager

- Fitted model: 20.0 ms fixed + 0.10 ms per 100 candidates. Latency is effectively independent of workload.
- Reducing the candidate count (iCEM) therefore yields no gain: predicted 1.01×, observed 0.92×.
- A gain requires an execution model with low fixed cost per call.

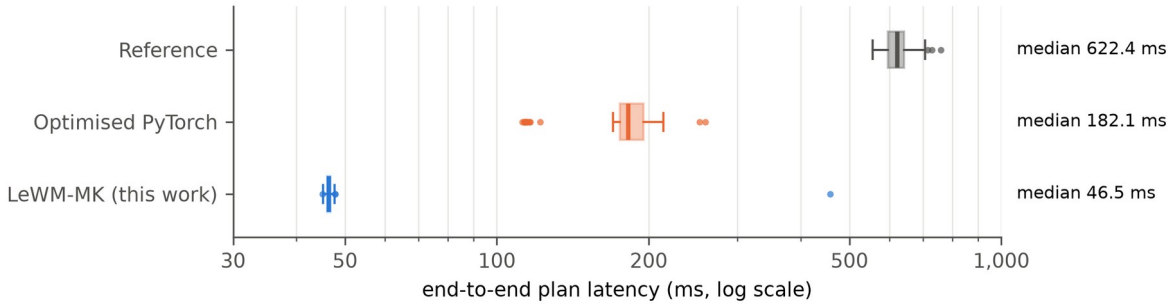
Solution design: LeWM-MK



Weights: 1,472 pre-swizzled 24 KB bf16 panels (with encoder). Each warpgroup's thread 0 refills its 2-slot shared-memory ring with cp.async.bulk from a static schedule, so weight prefetch runs through op, layer, pass and barrier boundaries.

- Encode each image once; cache the goal for the episode.
- iCEM search: 2,820 instead of 9,000 imagined futures.
- Whole plan in one persistent kernel: sampling, rollouts, scoring, elite selection, encoder; the CPU waits once.
- Small model spread across a 16-processor cluster; partial results exchanged on chip; barrier 3.6 → 0.65 µs.
- Weights streamed ahead of use; BF16 inputs, FP32 accumulation.

Results: planning latency



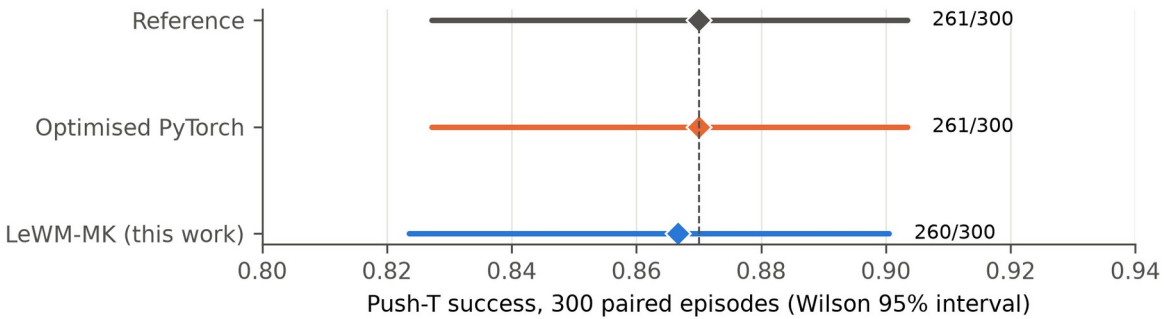
End-to-end planning latency, log scale: median, interquartile box, 1.5-IQR whiskers, outlying plans. Baseline uses compile, CUDA Graphs and BF16.

13.4×
vs reference
[13.0, 13.5]

3.9×
vs optimised
PyTorch

- 90th percentile 47.0 ms; interquartile range 0.7 ms wide.
- Fits a 20 Hz control loop at the median.
- Search now 46 µs per layer (was 409); synchronisation about 7% of it.

Results: task success and numerical fidelity



Push-T success on the same 300 episodes, Wilson 95% intervals. 300 pairs detect differences of about 6 points.

- 261 / 261 / 260 successes of 300 for reference, optimised PyTorch and LeWM-MK.
- LeWM-MK vs reference: 19 won, 20 lost; −0.3 points [−4.3, +3.7]; McNemar p = 1.00.
- Discordant episodes reflect stochastic search; the optimised PyTorch planner is discordant on 36.
- Numerical error vs FP32: 0.06–0.32%, matching accepted BF16 (0.05–0.40%).

Conclusions and limitations

01 The reference's time was overhead: 33% redundant encoding, a search 99% fixed cost per call, 7% CPU waits.

02 LeWM-MK removes each bottleneck with the model and search budget unchanged.

03 46.5 ms per plan: 13.4× the reference, 3.9× optimised PyTorch; success unchanged (−0.3 pts).

04 Planner plug-in; one program regenerates every result from the data.

Limitations

- 300 paired episodes resolve differences of about 6 percentage points.
- Some profile stages are attributed by difference.
- One model, one task, one GPU type; H100-only kernel.
- Simulation only; only the planning call was optimised.
- PyTorch latency varied by up to 18% between passes; kernel latency by about 2%.

Future work

1. **Search budget versus task success**
   Larger iteration and candidate budgets remain within a 100 ms control period; characterise the latency-success trade-off.

2. **Reducing the dependent computation chain**
   Combination with single-pass rollout models; layer-per-cluster pipelining.

3. **Generalisation to other models and hardware**
   Larger world models, embedded robot computers, automatic megakernel compilation.

4. **Higher-resolution and physical evaluation**
   Approximately 2,500 paired episodes to resolve 2 percentage points; evaluation on a physical robot.

Summary

- The reference planner's latency was dominated by execution overhead, not arithmetic.
- A linear latency-scaling model identified the bottleneck regime and predicted when iCEM reduces latency.
- A single persistent GPU kernel per plan reduces latency from 622 ms to 46.5 ms on the same H100, with the same model and no detectable change in task success.

Report, datasets and solution program: docs/paper and solution_pack/ in the project repository.